

SLATE User Manual

Strategy Learning & Autonomous Trading Engine

A Complete Guide to Discovering, Testing, and Evolving Trading Strategies

Version 3.0.0 - Updated June 27, 2026

What's New in Version 3.0.0

This version represents a **major architectural overhaul** with revolutionary autonomous capabilities:

- **Automatic Discovery System:** SLATE now starts with continuous discovery enabled - no manual intervention required
 - **Autonomous System:** Advanced AI-driven market exploration with self-directed trading decisions
 - **Startup Coordinator:** Intelligent system management with smart pause/resume based on user activity
 - **Enhanced Memory Systems:** Improved reflection memory, checkpoint recovery, and cross-cycle learning
 - **YouTube Integration:** Transcribe and analyze trading strategy videos for insight extraction
 - **Enhanced Safety:** All operations maintain paper-trading mode with realistic transaction costs
 - **Real-Time Performance:** 5-second discovery cycles with intelligent resource management
-

Table of Contents

1. Welcome to SLATE
 2. What SLATE Can Do For You
 3. Getting SLATE Running
 4. Understanding How SLATE Works
 5. Automatic Discovery System
 6. Autonomous System
 7. Using SLATE: Question & Answer Examples
 8. The Discovery System Explained
 9. Advanced Features: Memory Systems
 10. Advanced Features: YouTube Integration
 11. Finding Profitable Strategies: Practical Examples
 12. API Quick Reference
 13. Common Problems and Solutions
-

1. Welcome to SLATE

Welcome to SLATE - your personal AI research assistant for discovering profitable trading strategies.

What Exactly Is SLATE?

Think of SLATE as an automated research laboratory for trading strategies. Just as a pharmaceutical company uses robots to test thousands of chemical compounds to find effective medicines, SLATE uses artificial intelligence to test thousands of trading strategies to find ones that actually make money.

Here's the key difference: **SLATE never trades with real money**. It only does paper trading - simulating trades to see what would have happened. This means you can experiment freely without any financial risk.

Who Is SLATE For?

SLATE is designed for:

- **Traders** who want to find new strategies without spending months manually testing ideas
- **Data Scientists** who want a reliable backtesting environment with realistic market assumptions
- **Researchers** studying market behavior and strategy evolution
- **Students** learning about algorithmic trading and risk management
- **Anyone** curious about whether trading strategies can be discovered by AI

What “Paper Trading Only” Means

SLATE will never: - Connect to your exchange account - Execute real trades with your money - Require API keys for live trading

SLATE will: - Simulate trades using historical market data - Calculate realistic fees and slippage - Track what would have happened with real money - Help you learn without any financial risk

Major Architecture Changes in v3.0.0

Automatic Discovery: SLATE now starts with continuous discovery enabled. The moment you launch SLATE, it begins discovering strategies automatically.

Autonomous System: Advanced AI capabilities allow SLATE to make self-directed market exploration decisions during idle periods.

Smart Resource Management: The Startup Coordinator intelligently manages system resources, pausing discovery when you're working and resuming when idle.

2. What SLATE Can Do For You

SLATE automates the entire process of strategy research. Here's what it can do for you:

Automatic Strategy Discovery (Always On)

NEW IN v3.0: SLATE now runs continuous discovery automatically:

- Starts discovering strategies the moment you launch SLATE
- Runs in 5-second cycles continuously
- Tests diverse strategies across all timeframes (1m to daily)
- Pauses automatically when you make requests
- Resumes after 5 minutes of user inactivity

What this means for you: - Launch SLATE and it immediately starts working - No manual intervention required to begin discovery - Your requests always get priority over discovery - Maximum utilization of idle time for strategy discovery

Realistic Performance Testing

Many backtesting systems give overly optimistic results because they ignore trading costs. SLATE includes realistic market assumptions:

- **Trading fees:** 0.02% for maker orders, 0.05% for taker orders
- **Slippage:** 10-20 bps volatility-adjusted price movement
- **Partial fills:** 15% probability of partial fills
- **Fill rate:** 85% realistic fill rate (not 100%)
- **Position limits:** 5% max per position, 15% total portfolio exposure

Example Impact:

Without costs:

Entry: \$50,000

Exit: \$51,000

Profit: \$1,000 (2.0%)

With realistic costs:

Entry: \$50,000 (taker) → \$50,025 (fee + slippage)

Exit: \$51,000 (taker) → \$50,975.50 (fee - slippage)

Actual profit: \$950.50 (1.9%)

That's 5% less profit! And this compounds over many trades.

Autonomous Market Exploration

NEW IN v3.0: During idle periods, SLATE's autonomous system:

- Analyzes market conditions independently
- Generates trading goals and hypotheses
- Validates strategies with realistic costs
- Reports discoveries automatically
- Spawns specialized analysis agents

Safety Constraints: - Only operates during idle periods (5+ minutes of inactivity) - User queries immediately pause operations - All strategies validated with realistic transaction costs - Resource constraints enforced (CPU, memory, time) - Only modifies files within `slate_core/` directory

Multi-Path Stress Testing

SLATE doesn't just test a strategy once on historical data. It uses "bootstrap resampling" to create 100+ alternative price paths from the same historical data.

Think of it this way: Imagine you're testing a strategy on Bitcoin price data from 2023. But what if 2023 had played out slightly differently? SLATE creates 100 different "what if" versions of 2023 and tests your strategy on all of them.

If a strategy performs well on all 100 paths, it's robust. If it only works on one, it's lucky.

Advanced Memory Systems

SLATE has three sophisticated memory systems:

1. **GraphPalace (Knowledge Graph):** Remembers relationships between strategies and how they evolve
2. **Reflection Memory:** Learns from every cycle, extracting lessons and patterns
3. **Checkpoint Manager:** Crash recovery for incomplete discovery cycles

YouTube Integration

NEW IN v3.0: SLATE can now transcribe and analyze YouTube trading videos:

- Extract trading strategies from video content
- Search transcripts for specific concepts
- Cache transcripts for faster repeated access
- Identify insights from expert traders

3. Getting SLATE Running

What You Need

Hardware: - Any modern computer (Mac, Linux, or Windows) - 4GB of RAM minimum (8GB+ recommended) - 500MB of free disk space

Software: - Python 3.8 or higher - An internet connection (to fetch market data)

Installation in Three Steps

Step 1: Download SLATE Open your terminal and navigate to where you want SLATE to live:

```
cd ~/astrodata/SWARM/SLATE
```

Or if you're cloning from GitHub:

```
git clone https://github.com/Tilanthi/SLATE.git
cd SLATE
```

Step 2: Install Dependencies SLATE needs several Python libraries. Install them all at once:

```
pip install fastapi uvicorn numpy pandas scipy aiohttp ccxt pytest matplotlib
```

If you have a requirements.txt file:

```
pip install -r requirements.txt
```

Step 3: Verify Installation Run the test suite to make sure everything works:

```
python3 slate_core/run_tests.py
```

You should see something like:

```
===== 31 passed in 2.45s =====
```

All tests passing means SLATE is ready to go!

Starting SLATE

IMPORTANT: In v3.0, SLATE **automatically starts discovery** when launched. This is the default behavior.

Option 1: Start with the script (Recommended)

```
./start_slate.sh
```

Option 2: Start directly with Python

```
python3 -m slate_core.server
```

When SLATE starts, you'll see:

```
=====
SLATE Server Starting
=====
Port: 8788
Mode: Paper Trading Only
Dashboard: http://localhost:8788
API Docs: http://localhost:8788/docs
=====
INFO:      Initializing startup coordinator...
INFO:      Auto-starting discovery cycle...
INFO:      Discovery loop started - will run continuously
INFO:      Running automatic discovery cycles...
```

Discovery runs automatically in the background, continuously searching for profitable strategies.

Note: Port 8788 is used for SLATE. The server will auto-start discovery cycles - you don't need to manually trigger them.

Checking That SLATE Works

Let's verify everything is connected:

```
curl http://localhost:8788/health
```

You should get a response like:

```
{
  "status": "healthy",
  "mode": "paper_trading",
  "discovery_running": true,
  "startup_coordinator": {
    "state": "auto_discovery",
    "idle_time_seconds": 45.2,
    "resume_in_minutes": 4.2,
    "user_requested_pause": false
  }
}
```

This confirms SLATE is running in safe paper trading mode with automatic discovery active.

Verification Script

Run the built-in verification script:

```
python verify_startup.py
```

This tests: - Coordinator initialization - Initial state check - Discovery engine initialization - Status structure validation - Configuration verification

4. Understanding How SLATE Works

The Big Picture

SLATE consists of several interconnected components. Think of it like a research laboratory with different departments:

Let's walk through what each component does:

The Trading Engine (OODA Cycle)

SLATE uses a decision-making framework called OODA (Observe-Orient-Decide-Act):

Here's how it works:

1. **Observe:** SLATE watches the market and collects data
 - Current prices
 - Trading volume
 - Order book depth
 - Market events
2. **Orient:** SLATE analyzes the data
 - Calculates technical indicators (RSI, MACD, etc.)
 - Detects market conditions (trending vs ranging)
 - Assesses risk levels
 - Ranks available strategies

3. **Decide:** SLATE chooses what to do
 - Selects the best strategy for current conditions
 - Calculates how much to trade (position sizing)
 - Sets entry and exit points
 - Applies risk rules
4. **Act:** SLATE executes the trade (on paper)
 - Records the paper trade
 - Monitors the position
 - Tracks performance
5. **Learn:** SLATE updates its knowledge
 - Records what worked and what didn't
 - Updates strategy performance metrics
 - Adjusts parameters for next time

The Discovery System

This is where SLATE gets really interesting. The discovery system is SLATE's research department:

The discovery process:

1. **Strategy Generator:** Creates new strategies to test
 - 35+ different strategy types
 - Randomized parameters within sensible ranges
 - Multiple timeframes (1m to 1d)
2. **Realistic Backtester:** Tests each strategy honestly
 - Includes all trading fees
 - Models slippage realistically
 - Simulates partial fills
 - No artificial advantages
3. **Multi-Path Testing:** Stress tests each strategy
 - Creates 100+ price path scenarios
 - Tests strategy on each path
 - Keeps only strategies that work on most paths
4. **Self-Evolution Engine:** Improves strategies over time
 - Selects top performers
 - Creates optimized variants
 - Discards underperformers
 - Continuously improves the gene pool

The Memory Systems

SLATE has three sophisticated memory systems:

GraphPalace (Knowledge Graph): - Remembers relationships between strategies - Tracks how strategies evolve over time - Stores market context information - Like a "brain" that understands connections

Reflection Memory: - Logs every discovery cycle to markdown - Extracts lessons learned from performance - Provides context for future cycles - Human-readable format for transparency

Checkpoint Manager: - Saves progress after each strategy tested - Enables crash recovery - Stores incomplete cycle state - Prevents loss of long-running discoveries

The API Layer

SLATE provides a REST API that lets you interact with all these components. You can: - Start and stop discovery - Query results - Create custom strategies - Check system health - Export data for analysis - Control autonomous operations

5. Automatic Discovery System

The automatic discovery system is a **revolutionary feature** in SLATE v3.0 that ensures continuous strategy exploration without manual intervention.

How Automatic Discovery Works

1. **Startup Behavior:**
 - Discovery starts IMMEDIATELY when SLATE launches
 - No manual intervention required
 - System is “always on” by default
2. **Continuous Operation:**
 - Discovery runs in continuous cycles (5-second intervals)
 - Tests diverse strategies across all timeframes (1m to daily)
 - Stores results in persistent database
 - Learns from previous discoveries
3. **Smart Pause/Resume:**
 - User activity automatically pauses discovery
 - API calls, queries, and tasks trigger pause
 - Resumes after 5 minutes of user inactivity
 - Ensures user requests get priority
4. **Multi-Timeframe Exploration:**
 - Tests across: 1m, 5m, 15m, 30m, 1h, 4h, 8h, 12h, 1d
 - Ensures comprehensive temporal coverage
 - Discovers edges across different market rhythms

The Startup Coordinator

The `StartupCoordinator` class manages automatic discovery:

```
from slate_core.startup_coordinator import get_startup_coordinator

# Get coordinator (auto-starts discovery)
coordinator = get_startup_coordinator()

# Check status
status = coordinator.get_status()

# Returns: state, idle time, resume countdown, etc.
```



```
# Record user activity (auto-pauses)
coordinator.record_user_activity()
```

System States

The coordinator manages these states:

- **AUTO_DISCOVERY:** Running continuous discovery (default)
- **USER_TASK:** Executing specific user request
- **PAUSED:** Temporarily paused
- **IDLE:** Waiting to resume

Smart Pause/Resume Behavior

Automatic Pause Triggers: - Any API call to `/api/discovery/*` - Direct queries to the discovery engine - User-initiated strategy generation - Dashboard interactions

Automatic Resume: - Resumes after 5 minutes of no user activity - Checks system state before resuming - Ensures resources are available - Gracefully resumes discovery cycles

Integration Points

In Server (`slate_core/server.py`):

```
# Startup event initializes coordinator
startup_coordinator = get_startup_coordinator()

# All API endpoints track user activity
@app.post("/api/discovery/start")
async def start_discovery():
    track_user_activity() # Pauses discovery
    # ... user request handling
```

In Direct Usage:

```
from slate_core.startup_coordinator import execute_with_discovery_paused

# Execute task with discovery paused
result = await execute_with_discovery_paused(
    my_function, arg1, arg2
)
```

Benefits of Automatic Discovery

1. **Always On:** SLATE is continuously discovering strategies
2. **Smart Resource Management:** Pauses for user work, resumes when idle
3. **Better Utilization:** Maximizes discovery time
4. **User Priority:** User requests always get priority
5. **Improved Safety:** Better activity tracking and state management

Monitoring Automatic Discovery

Check status via API:

```
curl http://localhost:8788/health
```

Response includes:

```
{
  "startup_coordinator": {
    "state": "auto_discovery",
    "idle_time_seconds": 45.2,
    "resume_in_minutes": 4.2,
    "user_requested_pause": false
  }
}
```

Programmatic monitoring:

```
from slate_core.startup_coordinator import get_system_status
status = get_system_status()
print(f"Discovery running: {status['discovery_running']}")
print(f"State: {status['startup_coordinator']['state']}")
```

Customizing Automatic Discovery

```
from slate_core.startup_coordinator import get_startup_coordinator

coordinator = get_startup_coordinator()
coordinator.idle_timeout_minutes = 10 # Customize idle timeout
coordinator.discovery_cycle_interval_seconds = 10 # Customize interval
```

6. Autonomous System

The Autonomous System is **advanced AI capability** that allows SLATE to make self-directed market exploration decisions during idle periods.

What is the Autonomous System?

The Concept: Just as a human trader independently analyzes markets, forms hypotheses, and tests strategies, SLATE's autonomous system performs similar activities autonomously during idle periods.

How It Works: 1. **Idle Detection:** Activates after 5 minutes of user inactivity 2. **Market Analysis:** Independently analyzes current market conditions 3. **Goal Generation:** Creates trading goals and hypotheses 4. **Strategy Testing:** Validates strategies with realistic costs 5. **Discovery Reporting:** Reports findings automatically

The Autonomous Orchestrator

```
from slate_core.autonomous import AutonomousOrchestrator
```

```
orchestrator = AutonomousOrchestrator(config)
orchestrator.start() # Starts autonomous loop
```

Features: - Idle detection (activates after 5 minutes user inactivity) - Trading decision-making coordination - Strategy discovery and validation - Resource management and safety constraints - Reactive priority (user requests interrupt)

Safety Constraints

The autonomous system operates under strict safety constraints:

- **Only operates during idle periods** (5+ minutes of inactivity)
- **User queries immediately pause operations**
- **All strategies validated with realistic costs**
- **Resource constraints enforced** (CPU, memory, time)
- **Only modifies files within slate_core/ directory**
- **Maintains paper-trading mode only**

Autonomous Components

1. **ResourceManager:** Monitors CPU, memory, time usage
2. **TradingDecisionMaker:** Generates trading goals
3. **StrategyValidator:** Validates with realistic costs
4. **DiscoveryReporter:** Reports discoveries
5. **MarketSubAgentSpawner:** Spawns specialized agents

Using the Autonomous System

Start autonomous operations:

POST /api/autonomous/start

Stop autonomous operations:

POST /api/autonomous/stop

Get autonomous status:

GET /api/autonomous/status

Response:

```
{
  "status": "running",
  "idle_time_seconds": 345.2,
  "active_operations": 3,
  "discoveries_made": 127,
  "last_activity": "2026-06-27T10:30:00"
}
```

Get autonomous discoveries:

```
GET /api/autonomous/discoveries?limit=10
```

Generate autonomous report:

```
GET /api/autonomous/report
```

Example: Autonomous Discovery Session

```
# Start autonomous operations
```

```
curl -X POST http://localhost:8788/api/autonomous/start
```

```
# Go do something else for 30 minutes
```

```
# Check what was discovered
```

```
curl http://localhost:8788/api/autonomous/discoveries?limit=20
```

```
# Get a comprehensive report
```

```
curl http://localhost:8788/api/autonomous/report > autonomous_report.json
```

Benefits of the Autonomous System

1. **Continuous Exploration:** Works even when you're not actively using SLATE
2. **Independent Analysis:** Forms and tests its own hypotheses
3. **Resource Efficient:** Only runs during idle periods
4. **Safety First:** All operations maintain safety constraints
5. **Reactive Priority:** Immediately pauses for user requests

Autonomous vs. Manual Discovery

Manual Discovery: - You control when discovery runs - You specify parameters and constraints
- You initiate specific tests - Best for targeted exploration

Autonomous Discovery: - SLATE controls when to explore - SLATE forms its own hypotheses
- SLATE initiates tests independently - Best for broad exploration during idle time

They work together: Use manual discovery for targeted testing, autonomous discovery for broad exploration.

7. Using SLATE: Question & Answer Examples

The best way to understand SLATE is to see it in action. Here are common questions you might ask, and how SLATE answers them.

Q: "What's the best strategy for trading Bitcoin?"

How to ask SLATE:

```
curl -X POST http://localhost:8788/api/discovery/start \  
-H "Content-Type: application/json" \  

```

```
-d '{
  "cycles": 100,
  "symbols": ["BTCUSD"],
  "timeframes": ["1h"]
}'
```

What SLATE does: 1. Generates 100 different Bitcoin trading strategies 2. Tests each one on historical hourly data 3. Ranks them by actual profit (after fees) 4. Stores the results

How to get the answer:

```
curl http://localhost:8788/api/discovery/top?limit=5
```

Example answer:

```
[
  {
    "rank": 1,
    "strategy_name": "momentum_14_1h_btcusdt",
    "total_return": 0.234,
    "sharpe_ratio": 1.82,
    "max_drawdown": 0.18,
    "profit_factor": 2.1,
    "num_trades": 156
  },
  {
    "rank": 2,
    "strategy_name": "trend_follow_12_26_1h_btcusdt",
    "total_return": 0.198,
    "sharpe_ratio": 1.65,
    "max_drawdown": 0.22,
    "profit_factor": 1.9,
    "num_trades": 89
  }
]
```

Q: “What is SLATE currently doing?”

How to ask SLATE:

```
curl http://localhost:8788/health
```

Example answer:

```
{
  "status": "healthy",
  "mode": "paper_trading",
  "discovery_running": true,
  "startup_coordinator": {
    "state": "auto_discovery",
    "idle_time_seconds": 45.2,
    "resume_in_minutes": null,
  }
}
```

```

    "user_requested_pause": false
  },
  "autonomous_system": {
    "status": "idle",
    "active_operations": 0
  }
}

```

What this tells you: - SLATE is healthy - Running in paper trading mode (safe) - Automatic discovery is active - No user pause requested - Autonomous system is idle (waiting for 5 minutes of inactivity)

Q: “How many strategies has SLATE tested?”

How to ask SLATE:

```
curl http://localhost:8788/api/discovery/statistics
```

Example answer:

```

{
  "total_tests": 1247,
  "profitable_strategies": 312,
  "best_return": 0.342,
  "average_sharpe": 0.98,
  "discovery_cycles_completed": 42
}

```

Q: “What has the autonomous system discovered?”

How to ask SLATE:

```
curl http://localhost:8788/api/autonomous/discoveries?limit=10
```

Example answer:

```

[
  {
    "timestamp": "2026-06-27T10:30:00",
    "strategy_type": "momentum",
    "expected_return": 0.156,
    "confidence": 0.82
  },
  {
    "timestamp": "2026-06-27T10:28:00",
    "strategy_type": "mean_reversion",
    "expected_return": 0.098,
    "confidence": 0.67
  }
]

```

Q: “Generate a strategy from natural language”

How to ask SLATE:

```
curl -X POST http://localhost:8788/api/discovery/nl/generate \
-H "Content-Type: application/json" \
-d '{
  "description": "Test a mean reversion strategy when RSI is below 30",
  "provider": "openai"
}'
```

Example answer:

```
{
  "strategy_type": "mean_reversion",
  "parameters": {
    "rsi_period": 14,
    "rsi_threshold": 30,
    "stop_loss_atr": 2.0,
    "take_profit_atr": 3.0
  },
  "expected_return": 0.08,
  "explanation": "This strategy buys when RSI indicates oversold conditions..."
}
```

8. The Discovery System Explained

How SLATE Discovers Strategies

The discovery system is SLATE’s core innovation. Here’s how it works in detail:

Step 1: Strategy Generation

SLATE doesn’t just test random ideas. It generates intelligent strategy variations:

35+ Strategy Types:

1. **Momentum:** Bets that price movement will continue
2. **Mean Reversion:** Bets that price will return to average
3. **Breakout:** Bets that price will break through resistance
4. **Trend Following:** Uses moving averages to follow trends
5. **Statistical Arbitrage:** Exploits price relationships
6. **Machine Learning:** Uses ML to find patterns
7. **Regime Switching:** Changes behavior based on market conditions
8. **Order Flow:** Trades based on order book imbalances
9. **Market Microstructure:** Exploits short-term inefficiencies
10. **Time-Based:** Session patterns and time-of-day effects

Step 2: Realistic Backtesting

Each generated strategy gets tested with realistic assumptions:

Trading Costs: - Maker fee: 0.02% - Taker fee: 0.05% - Slippage: 10-20 bps (volatility-adjusted)
- Fill rate: 85% - Partial fills: 15% probability

This is why SLATE's realistic testing matters. Many strategies look great until you subtract fees, then they become unprofitable.

Step 3: Multi-Path Testing

SLATE goes beyond simple backtesting with bootstrap resampling:

The Multi-Path Solution: 1. Take the historical price data 2. Randomly shuffle the daily returns (keeping price behavior realistic) 3. Create 100+ new “alternate universe” price paths 4. Test your strategy on all of them 5. See if the strategy works in most universes

Good Strategy: - Works in 80+ out of 100 paths - Consistent performance across paths - Robust to different market sequences

Bad Strategy (Lucky): - Works in only 20-30 paths - Huge variance between paths - Probably just got lucky with one specific sequence

Step 4: Evaluation and Selection

After testing, SLATE scores each strategy:

Primary Metrics: - **USDT Profit:** Actual dollar profit (primary metric) - **Sharpe Ratio:** Return divided by risk (higher is better) - **Maximum Drawdown:** Worst peak-to-trough loss (lower is better)

Secondary Metrics: - **Win Rate:** Percentage of profitable trades - **Profit Factor:** Total wins divided by total losses - **Calmar Ratio:** Return divided by max drawdown

Selection Criteria: - USDT profit > \$500 - Sharpe ratio > 1.0 - Maximum drawdown < 30% - Works on 80%+ of price paths

Step 5: Evolution

The best strategies become “parents” for the next generation:

Mutation: - Take a good strategy (e.g., momentum with period 14) - Create variants with tweaked parameters (period 12, 13, 15, 16) - Test them all - Keep the ones that improve

Crossover: - Take two good strategies with different strengths - Combine their best features - Test the combination - Keep it if it's better than either parent

Over many generations, SLATE “discovers” better and better strategies.

9. Advanced Features: Memory Systems

SLATE has three sophisticated memory systems that work together to provide continuous learning and crash recovery.

GraphPalace (Knowledge Graph)

What it does: - Remembers relationships between strategies - Tracks how strategies evolve over time - Stores market context information - Like a “brain” that understands connections

How to use:

```
from slate_core.discovery.discovery_memory import get_discovery_memory
```

```
memory = get_discovery_memory()
memory.store_discovery(result)
```

Location: slate_core/palace_data/discoveries/

Reflection Memory

What it does: - Logs every discovery cycle to markdown - Extracts lessons learned from performance - Provides context for future cycles - Human-readable format for transparency

How to use:

```
from slate_core.discovery.reflection_memory import get_reflection_memory
```

```
memory = get_reflection_memory()
lessons = memory.get_recent_lessons(limit=10)
context = memory.get_context_for_new_cycle()
```

Location: ~/.slate/memory/discovery_memory.md

API Access:

```
# Get full reflection memory
```

```
curl http://localhost:8788/api/memory/reflection
```

```
# Get recent lessons
```

```
curl http://localhost:8788/api/memory/lessons?limit=10
```

```
# Get discovery context
```

```
curl http://localhost:8788/api/memory/context
```

```
# Clear reflection memory
```

```
curl -X POST http://localhost:8788/api/memory/clear
```

Checkpoint Manager

What it does: - Saves progress after each strategy tested - Enables crash recovery - Stores incomplete cycle state - Prevents loss of long-running discoveries

How to use:

```
from slate_core.discovery.checkpoint_manager import get_checkpoint_manager
```

```
mgr = get_checkpoint_manager()
incomplete = mgr.get_incomplete_cycles()
```

API Access:

Get checkpoint status

```
curl http://localhost:8788/api/discovery/checkpoint/status
```

Resume from checkpoint

```
curl -X POST http://localhost:8788/api/discovery/checkpoint/resume \  
  -H "Content-Type: application/json" \  
  -d '{"cycle_id": "abc123-def456"}'
```

Clear checkpoints

```
curl -X POST http://localhost:8788/api/discovery/checkpoint/clear
```

Location: ~/.slate/cache/checkpoints/

Benefits of Memory Systems

1. **Continuous Learning:** Each cycle builds on previous knowledge
 2. **Crash Recovery:** Never lose progress from long-running discoveries
 3. **Pattern Recognition:** Identifies what works across different market conditions
 4. **Adaptive Discovery:** Automatically shifts focus based on what's working
 5. **Transparency:** Human-readable format lets you understand SLATE's reasoning
-

10. Advanced Features: YouTube Integration

SLATE can now transcribe and analyze YouTube trading videos to extract insights and strategies.

What YouTube Integration Does

- **Transcribes Videos:** Converts spoken content to text
- **Searches Transcripts:** Finds specific concepts within videos
- **Extracts Insights:** Identifies trading strategies and ideas
- **Caches Results:** Stores transcriptions for faster access

Using YouTube Integration

Transcribe a video:

```
POST /api/youtube/transcribe
```

```
Content-Type: application/json
```

```
{  
  "video_id": "dQw4w9WgXcQ"  
}
```

Search a transcript:

```
POST /api/youtube/search
```

```
Content-Type: application/json
```

```
{
  "video_id": "dQw4w9WgXcQ",
  "query": "moving average crossover"
}
```

Check YouTube status:

GET /api/youtube/status

Clear transcript cache:

POST /api/youtube/cache/clear

Example: Extracting Strategy from Video

```
# Transcribe a video about trading strategies
curl -X POST http://localhost:8788/api/youtube/transcribe \
  -H "Content-Type: application/json" \
  -d '{"video_id": "xyz123"}'

# Search for specific concepts
curl -X POST http://localhost:8788/api/youtube/search \
  -H "Content-Type: application/json" \
  -d '{"video_id": "xyz123", "query": "RSI strategy"}'
```

Benefits

1. **Learn from Experts:** Extract insights from professional traders
2. **Strategy Discovery:** Find new strategies mentioned in videos
3. **Concept Search:** Quickly locate specific topics within videos
4. **Time Saving:** Cached transcriptions for repeated access

11. Finding Profitable Strategies: Practical Examples

Here are practical examples of how to use SLATE to find profitable trading strategies.

Example 1: Let Automatic Discovery Run

Goal: Let SLATE discover strategies automatically

Setup:

```
# Start SLATE - automatic discovery begins immediately
python -m slate_core.server
```

```
# Wait while SLATE discovers strategies
# Check back after an hour
```

Check results:

curl http://localhost:8788/api/discovery/top?limit=20

Result: A continuously updated list of discovered strategies ranked by performance.

Example 2: Use Autonomous System for Broad Exploration

Goal: Let SLATE explore markets autonomously while you do other things

Setup:

```
# Enable autonomous operations  
curl -X POST http://localhost:8788/api/autonomous/start
```

```
# Go do something else for 2-3 hours
```

Check what was discovered:

```
# Get autonomous discoveries  
curl http://localhost:8788/api/autonomous/discoveries?limit=20
```

```
# Get comprehensive report  
curl http://localhost:8788/api/autonomous/report > report.json
```

Result: Independent discoveries made by SLATE's autonomous system.

Example 3: Generate Strategy from Natural Language

Goal: Quickly test a strategy idea

Setup:

```
curl -X POST http://localhost:8788/api/discovery/nl/test \  
  -H "Content-Type: application/json" \  
  -d '{  
    "description": "Test a mean reversion strategy when RSI is below 30",  
    "provider": "mock"  
  }'
```

Result: Strategy generated and tested immediately.

Example 4: Analyze YouTube Trading Video

Goal: Extract strategies from a YouTube video

Setup:

```
# Transcribe the video  
curl -X POST http://localhost:8788/api/youtube/transcribe \  
  -H "Content-Type: application/json" \  
  -d '{"video_id": "youtube_video_id"}'  
  
# Search for specific strategies  
curl -X POST http://localhost:8788/api/youtube/search \  
  -H "Content-Type: application/json" \  
  -d '{"video_id": "youtube_video_id", "query": "entry strategy"}'
```

Result: Transcribed content with searchable strategy concepts.

Example 5: Build Diversified Portfolio

Goal: Combine multiple uncorrelated strategies

Setup:

```
# Let automatic discovery run for several hours  
# Then optimize portfolio  
curl http://localhost:8788/api/discovery/portfolio/optimize?method=mean_variance
```

Result: Optimal allocation weights for a diversified portfolio.

12. API Quick Reference

Base URL

All API calls go to:

`http://localhost:8788`

Interactive Documentation

- **Swagger UI:** `http://localhost:8788/docs`
- **ReDoc:** `http://localhost:8788/redoc`

System Health

Health Check

GET `/health`

Response:

```
{  
  "status": "healthy",  
  "mode": "paper_trading",  
  "discovery_running": true,  
  "startup_coordinator": {...},  
  "autonomous_system": {...}  
}
```

Discovery Control

Start Discovery

POST `/api/discovery/start`

Stop Discovery

POST `/api/discovery/stop`

Get Discovery Status

GET /api/discovery/status

Strategy Information

Get Top Strategies

GET /api/discovery/top?limit=10&sort_by=total_profit_usdt

Get Discovery Statistics

GET /api/discovery/statistics

Natural Language Strategy Generation

Generate Strategy from Description

POST /api/discovery/nl/generate

Content-Type: application/json

```
{
  "description": "Test a mean reversion strategy when RSI is below 30",
  "provider": "openai"
}
```

Generate and Test Strategy

POST /api/discovery/nl/test

Content-Type: application/json

```
{
  "description": "Test a breakout strategy when volume is high",
  "provider": "anthropic"
}
```

Autonomous System

Start Autonomous Operations

POST /api/autonomous/start

Stop Autonomous Operations

POST /api/autonomous/stop

Get Autonomous Status

GET /api/autonomous/status

Get Autonomous Discoveries

GET /api/autonomous/discoveries?limit=10

Generate Autonomous Report

GET /api/autonomous/report

YouTube Integration

Transcribe YouTube Video

POST /api/youtube/transcribe
Content-Type: application/json

```
{  
  "video_id": "youtube_video_id"  
}
```

Search Transcript

POST /api/youtube/search
Content-Type: application/json

```
{  
  "video_id": "youtube_video_id",  
  "query": "search term"  
}
```

Get YouTube Status

GET /api/youtube/status

Clear Transcript Cache

POST /api/youtube/cache/clear

Memory Systems

Get Reflection Memory

GET /api/memory/reflection

Get Recent Lessons

GET /api/memory/lessons?limit=10

Get Discovery Context

GET /api/memory/context

Clear Reflection Memory

POST /api/memory/clear

Checkpoint & Recovery

Get Checkpoint Status

GET /api/discovery/checkpoint/status

Resume from Checkpoint

POST /api/discovery/checkpoint/resume
Content-Type: application/json

```
{
  "cycle_id": "abc123-def456"
}
```

Clear Checkpoints

POST /api/discovery/checkpoint/clear
Content-Type: application/json

```
{
  "cycle_id": "abc123-def456"
}
```

Advanced Analytics

Benchmark Comparison

GET /api/discovery/benchmark

Strategy Correlation Analysis

GET /api/discovery/correlation

Portfolio Optimization

GET /api/discovery/portfolio/optimize?method=mean_variance

13. Common Problems and Solutions

Problem: “Port Already in Use”

Solution:

```
# Find what's using port 8788
lsof -ti:8788
```

```
# Kill that process
kill -9 $(lsof -ti:8788)
```

```
# Then restart SLATE
python3 -m slate_core.server
```

Problem: Discovery Not Starting

Possible causes:

1. No historical data available
 - SLATE will fetch data automatically on first run
 - This may take a few minutes
2. Database locked


```
# Remove lock files
rm -f slate_core/*.db-lock
rm -f slate_core/palace_data/*.db-lock
```

3. Startup coordinator issues

```
# Verify coordinator status
python -c "from slate_core.startup_coordinator import get_system_status; import pprint; pp
```

Problem: No Profitable Strategies Found

Possible causes:

1. **Market conditions:** Not all markets have profitable strategies
 - Try different timeframes
 - Try different symbols
 - Try different date ranges
2. **Too strict filters:** Relax criteria slightly
3. **Discovery hasn't run long enough:** Let automatic discovery run longer

Problem: Strategy Worked in Backtest, Fails in Paper Trading

This is normal and expected. Here's why:

- **Backtest** uses historical data (known, fixed)
- **Paper trading** uses live data (unknown, changing)

Solutions:

1. **Verify with multi-path testing:** Did it work on 80%+ of paths?
2. **Check for overfitting:** Extreme parameters often fail live
3. **Monitor regime changes:** Market conditions might have shifted
4. **Give it time:** Need at least 20+ trades to judge performance

Getting Help

If you encounter problems not covered here:

1. **Check the logs:** SLATE logs all activity to the terminal
 2. **Run the test suite:** `python3 slate_core/run_tests.py`
 3. **Check documentation:** Visit <http://localhost:8788/docs>
 4. **Run verification:** `python verify_startup.py`
-

Appendix

Glossary

Automatic Discovery: SLATE's continuous strategy discovery that runs automatically in the background

Autonomous System: Advanced AI that independently explores markets during idle periods

Alpha: Profit above what would be expected from market movement alone

Drawdown: The decline from a peak to a trough in your account value

Kelly Criterion: A formula for calculating the optimal position size based on win rate and reward/risk ratio

Maker/Taker Fees: Trading fees. Makers provide liquidity (cheaper), takers take liquidity (more expensive)

Multi-Path Testing: Testing a strategy on many alternative price paths to verify robustness

Paper Trading: Simulating trades without real money

Reflection Memory: Learning system that extracts lessons from past discovery cycles

Sharpe Ratio: A measure of risk-adjusted return. Higher is better. > 1.5 is good.

Slippage: The difference between the expected price of a trade and the actual price

Startup Coordinator: Manages automatic discovery and smart pause/resume functionality

Value at Risk (VaR): The maximum loss expected over a given time period at a given confidence level

Quick Reference

Server Information: - Default Port: 8788 - Mode: Paper trading only (never real money) - Initial Capital: 10,000 USDT (configurable)

Default Trading Costs: - Maker Fee: 0.02% - Taker Fee: 0.05% - Slippage: 10-20 bps (volatility-adjusted) - Fill Rate: 85% - Partial Fills: 15% probability

Default Risk Limits: - Max Position Size: 5% of capital per trade - Max Portfolio Heat: 15% total exposure - Max Drawdown: 25%

Common Commands:

```
# Start SLATE
```

```
python3 -m slate_core.server
```

```
# Run tests
```

```
python3 slate_core/run_tests.py
```

```
# Verify startup
```

```
python verify_startup.py
```

```
# Check health
```

```
curl http://localhost:8788/health
```

```
# Get top strategies
```

```
curl "http://localhost:8788/api/discovery/top?limit=10"
```

```
# Get autonomous status
```

```
curl "http://localhost:8788/api/autonomous/status"
```

```
# Start autonomous operations
```

```
curl -X POST http://localhost:8788/api/autonomous/start
```

Important URLs: - Main Dashboard: <http://localhost:8788> - Health Check: <http://localhost:8788/health>
- API Documentation: <http://localhost:8788/docs> - Discovery Statistics: <http://localhost:8788/api/discovery/statistics>
- Autonomous Status: <http://localhost:8788/api/autonomous/status> - Reflection Memory: <http://localhost:8788/api/memory/reflection>

Version: 3.0.0 **Last Updated:** June 27, 2026 **Mode:** PAPER TRADING ONLY - NEVER REAL MONEY

New in v3.0.0: - Automatic Discovery System (always on) - Autonomous System for self-directed exploration - Startup Coordinator for smart resource management - Enhanced Memory Systems (reflection, checkpoint, GraphPalace) - YouTube Integration for video analysis - 11 LLM providers for natural language strategy generation - Enhanced safety constraints and paper trading mode

IMPORTANT: SLATE is for research and education only. It never executes real trades. Any strategy discovered by SLATE should be thoroughly validated with extended paper trading before considering real-money implementation. Past performance does not guarantee future results.

For the latest updates, documentation, and source code, visit: <https://github.com/Tilanthi/SLATE>

Questions or issues? <https://github.com/Tilanthi/SLATE/issues>

Happy strategy hunting!